جامعة الإمارات العربية المتحدة
United Arab Emirates University

Software Engineering
Research Proposal
Course Code: CSPG 751
Semester: Fall 2026

Submitted by: Maryam Hanif (700052696)
Hani Abusharkh (700052516)

Sprint Risk Prediction from Reconstructed Jira Histories

1. Introduction

Scrum teams work in sprints, fixed cycles in which the team commits to a set of issues and tries to finish them by a planned end date. Two problems come up again and again. Some committed issues are not finished and spill over into the next sprint, and some sprints are closed late, after the date they were supposed to end. In most teams these problems are noticed only near the end of the sprint, when there is little time left to react. This project asks whether the historical data a team already has in Jira can predict these two outcomes before the sprint starts, and how much is gained by also watching the sprint while it is in progress. Both outcomes can be read directly from Jira. A sprint has spillover if at least one committed issue is still open at the planned end, and it is closed late if its completion is recorded more than one day after the planned end. The work is divided into three phases. Phase 1 predicts both outcomes at planning time. Phase 2 adds numeric targets (issues completed, carry share and days late), explains the predictions with exact Shapley values, and builds a what-if advisor. Phase 3 makes the prediction during the sprint, at the 25, 50, 75 and 100 percent checkpoints. The main risk throughout is data leakage: projects run several sprints at the same time, so a history feature built without care can use an outcome that was not yet known at planning. We therefore apply strict timing so that every feature uses only information available at the sprint start.

2. Literature Review

Effort and velocity estimation

- Choetkiertikul et al. [1]: deep learning to estimate story points from issue text.
- Porru et al. [2]: story points estimated from the fields of an issue report.
- Tawosi et al. [3]: replication of the deep-learning work on a shared dataset; simple baselines matched it once the feature set was fixed.
- Sprint2Vec [4]: sprint embeddings learned from issue-level data; predicts delivered volume and quality.
- Hearty et al. [5]: velocity forecasting with a dynamic Bayesian network.

- Gap: all of these predict effort or volume delivered. None predicts spillover or late closure, and none rebuilds the sprint-start state from the change history.

Dataset

- TAWOS [6]: public, multi-project Jira dataset built for reproducible studies; the data source for this work.

Sprint-level classification (motivates our leakage checks)

- Perez Castillo et al. [7]: sprint performance classified from burndown features that exist only while the sprint runs; retrospective rather than predictive.
- Obike et al. [8]: near-perfect deliverability AUC after SMOTE [15], with no clear train-test split before resampling. We reproduce both mechanisms on our own data to measure what each one costs.

Explainability

- Lundberg and Lee [9]: Shapley values as a model-agnostic attribution method.
- Jiarpakdee et al. [11]: correlated software metrics distort per-feature interpretations; this is why we compute exact Shapley values over feature groups.
- Molnar [10]: general treatment of interpretable machine learning.

In-sprint prediction

- Teinemaa et al. [12]: outcome-oriented predictive process monitoring, where a running case is scored at successive prefixes.
- Fahrenkrog-Petersen et al. [13]: alarm-based extensions of the same idea.
- Gap: not yet applied to Scrum sprint data with a reconstructed sprint state.

Concept drift

- Gama et al. [14]: drift as a reason a model fitted on one period or project fails on another; we test for it directly.

3. Contribution

- **C1.** Planning-time classifiers for spillover and late closure, trained on the reconstructed sprint-start state and evaluated with per-project time splits, leave-one-project-out (LOPO) testing, and project-cluster bootstrap intervals.
- **C2.** Exact group-level Shapley explanations plus a suite of simple baselines that any learned model must beat.
- **C3.** A sprint advisor that returns 80 percent prediction intervals and what-if tables, together with a test of whether the what-if tables carry any causal content.
- **C4.** In-sprint early warning at 25, 50, 75 and 100 percent of the planned duration, compared with the same model without progress features and with a learning-free progress rule.
- **Core method.** A leakage discipline for sprint data: scope reconstructed from the change log; strict timing on every history feature; zero-scope sprints removed; two reproduced leakage case studies showing how one careless feature or one misplaced resampling step inflates results.

4. Research Questions

- **RQ1.** How much predictive signal exists for spillover and late closure at planning time, and does committed scope alone explain most of it?

- **RQ2.** Do exact Shapley explanations confirm that spillover is driven by scope while late closure is driven by team habits such as past lateness?
- **RQ3.** Does observing in-sprint progress improve prediction, and if so, for which outcome and how early?
- **RQ4.** Are the results stable across time splits and across projects, or do they shift with concept drift?

5. What We Will Do

5.1 Data

Item	Detail
Source	TAWOS [6]: Sprint, Issue and Change_Log tables
Projects	8 open-source Jira projects, each with at least 30 sprint records: Apache Mesos, Appcelerator Studio, Atlassian Confluence Cloud and Server, LSST Data Management, Mule, Titanium SDK, Titanium Mobile
Sprints	2,602 modeled sprints; 1,691 of them with committed scope
Reconstruction	Sprint membership and story points rebuilt from the timestamped change log, as they stood on each sprint's first day, rather than read from the final Jira export
Exclusions	Sprints never closed (FUTURE or ACTIVE); sprints whose completion precedes their start; the first eligible sprint of each project (cold start)

5.2 Phase 1: Planning-Time Classification

Phase 1 predicts spillover and late closure before the sprint starts, using only the information available at planning time. It is the foundation for the rest of the project, since Phases 2 and 3 reuse the same reconstructed data and the same evaluation splits. The reference point is a very simple model based on scope alone, and any richer model has to show that its extra complexity is worth it.

- **Models:** logistic regression; random forest [17]; gradient-boosted trees [16].
- **Planning-time features:** committed scope (issue count, story points); recent history; sprint length; project identity.
- **Targets:** spillover; late closure.
- **Evaluation:** per-project 70/30 time split; leave-one-project-out cross-validation.
- **Comparators:** scope-only logistic model; project base-rate rule.

5.3 Phase 2: Explanation, Numeric Targets and Advisor

Phase 2 moves from yes-or-no labels to numbers, and from predictions to explanations. It also corrects the timing of the history features so that only sprints already finished by the start date are used. The models are then packaged into a small advisor, and we check whether its what-if tables have any causal meaning or only describe the fitted model.

- **Strict history features:** only sprints whose outcome was already settled on the start date.
- **Numeric targets:** issues completed; carry share; days late (capped at 30).
- **Explanations:** exact Shapley values computed per feature group.
- **Advisor:** 80 percent prediction intervals; what-if tables over alternative planned scopes.

- **Causal check:** what-if tables compared against realized carryover and lateness.

5.4 Phase 3: In-Sprint Early Warning

Phase 3 changes when the prediction is made, moving it from before the sprint to during it. The state of each sprint is rebuilt at fixed checkpoints using only the events recorded up to that point. The question is how much the progress information adds to the planning-time prediction, for which outcome, and how early in the sprint it becomes useful.

- **Checkpoints:** 25, 50, 75 and 100 percent of the planned duration; only events time-stamped at or before each checkpoint.
- **Progress features:** work done so far; remaining scope; time pressure; scope added or removed since the start.
- **Comparison:** same model without progress features; learning-free progress rule.
- **Exclusion:** sprints already closed at a checkpoint (outcome already known).

5.5 Baselines and Measures

Target	No-learning baseline to beat
Spillover	Always predict spillover (base rate about 94 percent)
Late closure	Always predict late
Issues completed	Committed scope times the team's historical completion rate
Carry share; days late	Training-set median
In-sprint (Phase 3)	Share of committed issues still open (spillover, carryover); number of open issues (late closure)

Aspect	Measure
Binary outcomes	ROC-AUC (primary); F1; precision; recall; PR-AUC; Brier score
Numeric outcomes	MAE; R-squared; pinball loss for quantile forecasts
Calibration	Predicted probability vs. observed rate, in quartile bins
Uncertainty	Bootstrap over the 8 projects, 2,000 resamples, under both the time split and LOPO
Advisor intervals	Coverage and width of the 80 percent intervals
Compute	Fixed seed; single laptop CPU; no GPU; under 20 minutes end to end; Python with scikit-learn [18] and XGBoost [16]

6. Threats to Validity

- **Internal:** leakage through history features that use future outcomes. Mitigation: strict timing on every feature; Phase 3 checkpoint reconstruction verified against Phase 1 final counts for all 2,602 sprints.
- **External:** all 8 projects are open-source Jira repositories; results may not transfer to proprietary or enterprise teams. Late closure depends on when someone clicks complete sprint, which varies by team culture.

- **Construct:** Shapley values describe the fitted model, not causal effects; what-if tables show model expectations, not what would happen if scope changed. An issue resolved, reopened and resolved again is counted at its last resolution, which can make early progress look slower but never leaks the future.

7. Weekly Plan

Five weeks from the start of the project; final week reserved for leakage case studies, validation and writing.

Week	Dates (2026)	Phase, activities and deliverable
Week 1	26 Oct to 1 Nov	Phase 1. Reconstruct sprint-start state from the TAWOS change log; build planning-time features; train and evaluate Phase 1 classifiers (time split, LOPO). Deliverable: reconstruction pipeline; Phase 1 results table.
Week 2	2 Nov to 8 Nov	Phase 1 to 2. Add strict history features; confirm the parallel-sprint timing fix; set up the three numeric targets and baselines. Deliverable: strict-feature results; numeric-target baselines.
Week 3	9 Nov to 15 Nov	Phase 2. Exact group-level Shapley values for all models; advisor with prediction intervals and what-if tables; causal check of the what-if tables. Deliverable: explanation figures; advisor evaluation.
Week 4	16 Nov to 22 Nov	Phase 3. Rebuild sprint state at the four checkpoints; train progress-aware models; compare with planning-only models and the learning-free rule. Deliverable: in-sprint checkpoint table.
Week 5	23 Nov to 29 Nov	Integration. Reproduce the two leakage case studies; verify checkpoint counts against Phase 1; assemble results; write the report; package the artifact. Deliverable: final report; reproducible artifact.

8. References

[1] M. Choetkiertikul, H. K. Dam, T. Tran, T. Pham, A. Ghose, and T. Menzies, “A deep learning model for estimating story points,” IEEE Trans. Softw. Eng., vol. 45, no. 7, pp. 637–656, Jul. 2019.

[2] S. Porru, A. Murgia, S. Demeyer, M. Marchesi, and R. Tonelli, “Estimating story points from issue reports,” in Proc. 12th Int. Conf. Predictive Models Data Analytics Softw. Eng. (PROMISE), 2016, pp. 1–10.

[3] V. Tawosi, R. Moussa, and F. Sarro, “Agile effort estimation: Have we solved the problem yet? Insights from a replication study,” IEEE Trans. Softw. Eng., vol. 49, no. 4, pp. 2677–2697, Apr. 2023.

[4] M. Choetkiertikul, P. Banyongrakkul, C. Ragkhitwetsagul, S. Tuarob, H. K. Dam, and T. Sunetnanta, “Sprint2Vec: A deep characterization of sprints in iterative software development,” IEEE Trans. Softw. Eng., vol. 51, no. 1, pp. 220–242, Jan. 2025.

[5] P. Hearty, N. Fenton, D. Marquez, and M. Neil, “Predicting project velocity in XP using a learning dynamic Bayesian network model,” IEEE Trans. Softw. Eng., vol. 35, no. 1, pp. 124–137, Jan. 2009.

[6] V. Tawosi, A. Al-Subaihin, R. Moussa, and F. Sarro, “A versatile dataset of agile open source software projects,” in Proc. 19th Int. Conf. Mining Softw. Repositories (MSR), 2022, pp. 707–711.

[7] Y. J. Perez Castillo, S. D. Orantes Jimenez, and P. O. Letelier Torres, “Sprint management in Agile approach: Progress and velocity evaluation applying machine learning,” Information, vol. 15, no. 11, Art. no. 726, 2024.

[8] P. G. Obike, V. E. Ekong, and O. U. Obot, “A novel scoring-based Agile sprint deliverability prediction and prioritization using machine learning,” Int. J. Comput. Appl., vol. 187, no. 49, pp. 40–53, Oct. 2025.

[9] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems 30 (NIPS 2017), 2017, pp. 4765–4774.

[10] C. Molnar, Interpretable Machine Learning: A Guide for Making Black Box Models Explainable, 2nd ed., 2022.

[11] J. Jiarpakdee, C. Tantithamthavorn, and A. E. Hassan, “The impact of correlated metrics on the interpretation of defect models,” IEEE Trans. Softw. Eng., vol. 47, no. 2, pp. 320–331, Feb. 2021.

[12] I. Teinemaa, M. Dumas, M. La Rosa, and F. M. Maggi, “Outcome-oriented predictive process monitoring: Review and benchmark,” ACM Trans. Knowl. Discov. Data, vol. 13, no. 2, Art. no. 17, 2019.

[13] S. A. Fahrenkrog-Petersen et al., “Fire now, fire later: Alarm-based systems for prescriptive process monitoring,” Knowl. Inf. Syst., vol. 64, no. 2, pp. 559–587, 2022.

[14] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, “A survey on concept drift adaptation,” ACM Comput. Surv., vol. 46, no. 4, Art. no. 44, 2014.

[15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” J. Artif. Intell. Res., vol. 16, pp. 321–357, 2002.

[16] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2016, pp. 785–794.

[17] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[18] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.